

Lenguajes de Programación y Transducción

Terra: documentación de las reglas semánticas

Segunda entrega — Semántica y procesamiento de datos

Yslen Natalia Moreno Gutierrez
Santiago Patiño Sarmiento

Universidad Sergio Arboleda
Escuela de ciencias exactas e ingeniería
Bogotá D.C., 20 de octubre de 2026

Índice

1. Propósito de este documento **4**

2. Sistema de tipos **4**

2.1. El tipo tabla 4

2.2. Reglas sobre los nulos 5

3. Contexto de ejecución y tabla de símbolos **5**

3.1. Estructura 5

3.2. Reglas de los identificadores 5

3.3. Ámbitos temporales 6

4. Carga y guardado de datos **6**

4.1. cargar 6

4.2. guardar 8

5. Tuberías **9**

5.1. seleccionar 9

5.2. filtrar 9

5.3. crear 10

5.4. renombrar 10

5.5. ordenar 11

5.6. eliminar duplicados 11

5.7. eliminar nulos 11

5.8. rellenar nulos 11

5.9. convertir 12

5.10. limitar 12

6. Agrupamiento y agregaciones **13**

6.1. agrupar 13

6.2. resumir 13

6.3. Funciones de agregación 14

7. Estadística descriptiva: describir **14**

8. mostrar con tablas **15**

9. Funciones como valores **15**

9.1. Funciones anónimas 15

9.2. Funciones con nombre como valores 15

9.3. Funciones de orden superior 16

10. Matemáticas y matrices **16**

10.1. Funciones de una variable 17

10.2. Vectores y matrices 17

10.3. Número de argumentos 17

11. Gestor de memoria y recursión **18**

11.1. Límite de recursión 18

11.2. Consulta y liberación 18

12. Entrada por teclado **18**

13. Tablas, listas y archivos **19**

13.1. De tabla a lista y matriz 19

13.2. Combinar tablas 19

13.3. Archivos de texto 19

13.4. Secuencias de escape en los textos 20

13.5. Tuberías dentro de argumentos 20

14. Ayuda integrada **20**

15. Formato de los errores **20**

15.1. Resumen de las categorías de error 21

1. Propósito de este documento

La gramática dice qué programas están *bien escritos*. La semántica dice qué *significan*: qué valor produce cada construcción, qué tipos acepta y en qué casos un programa que está bien escrito igual no se puede ejecutar.

Por ejemplo, esta línea es correcta para la gramática:

```
fijo r = datos |> seleccionar [temperatura]
```

Pero si la tabla `datos` no tiene una columna `temperatura`, el programa no tiene sentido. Detectar eso es trabajo del análisis semántico, que en Terra hace el **Visitor** (`VisitanteEvaluador`) mientras recorre el árbol de análisis.

Este documento describe, instrucción por instrucción, las reglas que aplica Terra en la segunda entrega. Para cada una se indica:

- **Forma**: cómo se escribe.
- **Recibe**: qué tipos acepta cada parte.
- **Devuelve**: el valor que produce.
- **Efecto**: qué cambia al ejecutarse.
- **Errores**: los casos que Terra rechaza y el mensaje que muestra.

Las reglas del primer corte (variables, expresiones, control de flujo y funciones) se mantienen igual; aquí se resumen en la sección 3 y se detallan las nuevas.

2. Sistema de tipos

Terra es un lenguaje de **tipado dinámico pero estricto**: las variables no declaran su tipo, pero cada valor tiene uno y las operaciones verifican que los tipos sean compatibles antes de ejecutarse. Nunca se convierte un tipo en otro de forma silenciosa.

Tipo	Valores	Ejemplo
`numero`	Enteros	`2023`
`decimal`	Números con parte decimal	`86.2`
`texto`	Cadenas entre comillas dobles	`"Colombia"`
`booleano`	`verdadero` o `falso`	`verdadero`
`nulo`	Ausencia de valor	`nulo`
`lista`	Secuencia de valores	`[1, 2, 3]`
`tabla`	Datos en filas y columnas (**nuevo**)	`cargar "x.csv"`
`funcion`	Una función usada como valor (**nuevo**)	`funcion(x) => x * 2`

2.1. El tipo tabla

Una tabla tiene una lista ordenada de **columnas** con nombres distintos y una lista de **filas**. Cada fila tiene exactamente un valor por columna, y cada valor (celda) es de tipo `numero`, `decimal`, `texto`, `booleano` o `nulo`. Una celda no puede contener una lista ni otra tabla.

El tipo de una columna se calcula a partir de sus valores no nulos:

- Si todos son del mismo tipo, la columna es de ese tipo.
- Si mezcla `numero` y `decimal`, se considera `decimal`.
- Si todos son nulos, la columna es de tipo `nulo`.
- Cualquier otra mezcla se reporta como `mixto`.

Regla de inmutabilidad. Ninguna operación modifica una tabla existente: todas devuelven una tabla nueva. Si `datos` se filtra, `datos` sigue teniendo todas sus filas.

> **Por qué:** Así una operación nunca altera en silencio un resultado que se calculó antes. Cada variable conserva exactamente el valor que tenía al declararse.

2.2. Reglas sobre los nulos

Terra trata el `nulo` de dos maneras distintas según el contexto:

1. **En operaciones aritméticas es un error.** `co2 + 1` con un `co2` nulo produce un error de tipos. Esto obliga a decidir qué hacer con los datos faltantes (eliminarlos o rellenarlos) en vez de que contaminen los cálculos sin que nadie lo note.
2. **En las agregaciones se ignora.** `media(co2)` dentro de `resumir` calcula la media de los valores presentes, como se hace en estadística descriptiva.

3. Contexto de ejecución y tabla de símbolos

3.1. Estructura

El Visitor guarda los identificadores en un diccionario llamado `memoria`. Cada nombre apunta a su valor y a si se puede reasignar:

```
{"datos":   {"valor": <Tabla 16 filas x 5 columnas>, "mutable": False},
 "umbral":  {"valor": 50, "mutable": True}}
```

En esta entrega la tabla de símbolos asocia identificadores con **objetos de datos**: una variable puede guardar una tabla completa, igual que guarda un número.

Las funciones definidas por el usuario se guardan aparte, en el diccionario `funciones`, y las incorporadas (`media`, `describir`, ...) en un registro fijo del evaluador.

3.2. Reglas de los identificadores

Regla	Mensaje de error
Un identificador debe declararse antes de usarse	`la variable 'x' no ha sido declarada`

Regla	Mensaje de error
No se puede declarar dos veces el mismo nombre	`la variable 'x' ya fue declarada`
Una variable `fijo` no se puede reasignar	`'x' se declaro con 'fijo' y no puede reasignarse`
Una función se define una sola vez	`la funcion 'f' ya fue definida`
Las funciones incorporadas no se redefinen	`'media' es una funcion incorporada del lenguaje y no puede redefinirse`
Solo se llaman funciones existentes	`la funcion 'f' no existe`

3.3. Ámbitos temporales

Además del ámbito global, hay tres situaciones en las que se crean nombres temporales. En todas, al terminar se **restaura** la memoria tal como estaba:

1. **La variable de un ciclo `para`**, que solo existe dentro del ciclo.
2. **Las columnas dentro de `filtrar` y `crear`**. Para cada fila, cada columna usada en la expresión se declara como variable no reasignable con el valor de esa fila. Por eso se puede escribir `filtrar donde co2 > 50`.
3. **Las columnas dentro de `resumir`**. Para cada grupo, cada columna usada se declara como la *lista* de sus valores no nulos en ese grupo. Por eso `suma(co2)` funciona con la misma función `suma` que se usa para cualquier lista.

Si una columna tiene el mismo nombre que una variable del programa, dentro de la expresión de la operación la columna *oculta* a la variable. Las variables cuyo nombre no coincide con ninguna columna siguen siendo visibles, así que se pueden combinar:

```
fijo umbral = 80
fijo altos = datos |> filtrar donde co2 > umbral
```

> **Por qué:** Solo se declaran las columnas que realmente aparecen en la expresión. El Visitor revisa una vez el subárbol de la expresión, guarda los identificadores que contiene y en cada fila declara solo esos. Con una tabla de 79 columnas y 50 000 filas, esto evita millones de asignaciones innecesarias.

4. Carga y guardado de datos

4.1. cargar

Forma	`cargar ruta [con separador s] [con encabezado b]`
Recibe	`ruta`: texto. `s`: texto no vacío. `b`: booleano.
Devuelve	Una tabla.
Efecto	Lee el archivo; no modifica nada del programa.

Valores por defecto: separador `","` y encabezado `verdadero`. Las opciones pueden ir en cualquier orden. Sin encabezado, las columnas se llaman `col1`, `col2`, ...

Lectura del archivo. El contenido se divide en registros con un autómata de dos estados (dentro o fuera de comillas):

- Fuera de comillas, el separador termina un campo y el salto de línea termina un registro.
- Dentro de comillas, el separador y el salto de línea son texto normal, y dos comillas seguidas (`""`) representan una comilla.
- Se ignora la marca BOM que agrega Excel y los finales de línea de Windows.
- Las líneas en blanco no son registros.

Si el archivo no tiene ninguna comilla, cada línea se parte directamente por el separador; el resultado es el mismo y la lectura es más rápida.

Inferencia de tipos. El tipo se decide *por columna*, probando en este orden:

1. `booleano`, si todos los valores no nulos son `verdadero`, `falso`, `true` o `false`;
2. `numero`, si todos son enteros;
3. `decimal`, si todos son números;
4. `texto` en cualquier otro caso.

Las celdas vacías y los textos `NA`, `N/A`, `NaN`, `null`, `nulo` y `none` (sin importar mayúsculas) se leen como `nulo`.

> **Por qué:** Si se decidiera celda por celda, una columna de códigos como `001`, `A12` quedaría con números en unas filas y textos en otras, y cualquier operación sobre ella fallaría de forma impredecible.

Errores:

Situación	Mensaje
Ruta que no es texto	`la ruta de 'cargar' debe ser texto, se recibio numero`
Separador que no es texto	`el separador debe ser texto, se recibio ...`
Encabezado que no es booleano	`la opcion 'encabezado' debe ser verdadero o falso, se recibio ...`
Opción repetida	`la opcion 'separador' aparece repetida en 'cargar'`
Archivo inexistente	`no se encontro el archivo '...'`
La ruta es una carpeta	`'...' es una carpeta, no un archivo`
Sin permiso de lectura	`no hay permiso para leer '...'`
Codificación distinta de UTF-8	`el archivo '...' no esta codificado en UTF-8`
Archivo vacío	`el archivo '...' esta vacio`

Situación	Mensaje
Columna del encabezado sin nombre	`la columna 3 del encabezado no tiene nombre`
Columnas repetidas	`la columna 'x' esta repetida`
Fila con más o menos campos	`la linea 7 del archivo tiene 6 campos y el encabezado tiene 5...`
Comillas sin cerrar	`hay comillas sin cerrar en el registro que empieza en la linea ...`
Separador equivocado	`el archivo parece usar ',' como separador, no ';'. Revisa la opcion 'con separador'`

La última regla se activa cuando cada registro queda como un solo campo que contiene otro separador común (`,`, `;`, tabulador o `|`). Sin ella, Terra cargaría una tabla de una sola columna y el error aparecería varias líneas después con un mensaje confuso.

4.2. guardar

Forma	`guardar tabla como ruta [con separador s]`
Recibe	`tabla`: tabla. `ruta`: texto. `s`: texto (por defecto `","`).
Devuelve	Nada (es una sentencia).
Efecto	Crea o reemplaza el archivo CSV indicado.

El archivo se escribe en UTF-8, con encabezado y separado por comas o por el separador indicado. Los nulos quedan como celdas vacías y los booleanos como `verdadero` y `falso`. Un campo se encierra entre comillas si contiene el separador, comillas o saltos de línea. Así, un archivo guardado por Terra se puede volver a cargar y produce exactamente la misma tabla.

Errores:

Situación	Mensaje
El valor no es una tabla	`'guardar' necesita una tabla, se recibio numero`
La ruta no es texto	`la ruta de 'guardar' debe ser texto, se recibio ...`
El separador no es texto	`el separador debe ser texto, se recibio ...`
La carpeta de destino no existe	`no se pudo guardar '...': la carpeta de destino no existe`
Sin permiso de escritura	`no hay permiso para escribir en '...'`

5. Tuberías

Forma	`expr |> operacion |> operacion ...`
Recibe	Una tabla al inicio.
Devuelve	La tabla que produce la última operación.
Efecto	Ninguno: cada operación crea una tabla nueva.

Las operaciones se aplican de izquierda a derecha: cada una recibe la tabla que produjo la anterior. Una expresión sin `|>` se evalúa como una expresión normal.

Reglas generales:

Regla	Mensaje
El valor inicial debe ser una tabla	`una tuberia '|>' debe empezar con una tabla, se recibio numero`
Después de `agrupar` solo puede venir `resumir`	`despues de 'agrupar' debe venir 'resumir'`
Una tubería no puede terminar en `agrupar`	`la tuberia termina en 'agrupar': falta 'resumir'`
Toda columna nombrada debe existir	`la columna 'x' no existe. Columnas disponibles: ...`

El mensaje de columna inexistente lista las columnas disponibles para que el error se corrija sin tener que abrir el archivo. Todos los errores de una operación se reportan con la línea donde está esa operación.

5.1. seleccionar

Forma	`|> seleccionar [c1, c2, ...]`
Recibe	Nombres de columnas existentes, sin repetir.
Devuelve	Una tabla con solo esas columnas, en el orden indicado.
Efecto	Las filas se conservan completas.

Errores: columna inexistente; columna repetida en la lista (`la columna 'x' esta repetida`).

5.2. filtrar

Forma	`|> filtrar donde condicion`
Recibe	Una expresión que, evaluada en cada fila, da un booleano.
Devuelve	Una tabla con las filas para las que la condición es `verdadero`.
Efecto	Las columnas conservan su orden; las filas, su orden original.

La condición se evalúa una vez por fila, con las columnas usadas declaradas como variables temporales (sección 3). Admite todos los operadores del lenguaje, incluidos `&&`, `||`, `!` y la pertenencia `en`:

```
|> filtrar donde pais en ["Colombia", "Peru"] && anio >= 2015
```

Errores:

Situación	Mensaje
La condición no es booleana	`la condicion de 'filtrar' debe ser booleana, se obtuvo decimal`
Tipos incompatibles en la condición	El error de tipos del operador, por ejemplo al operar con un nulo
Nombre que no es columna ni variable	`la variable 'x' no ha sido declarada`

> **Por qué:** No se acepta un número como condición (como hace Python con `0` y `1`). `filtrar donde co2` es casi siempre un error de quien escribe el programa, y es mejor señalarlo que adivinar la intención.

5.3. crear

Forma	`|> crear nombre = expresion`
Recibe	Una expresión que, evaluada en cada fila, da un valor de celda.
Devuelve	La tabla con la columna nueva al final, o con la columna reemplazada si ya existía.
Efecto	Ninguno sobre la tabla original.

Errores:

Situación	Mensaje
El resultado no es un valor de celda (por ejemplo, una lista)	`la columna 'x' solo puede guardar numeros, decimales, textos, booleanos o nulos`
Operación con un nulo	El error de tipos del operador
División por cero	El error del operador `/` definido en el primer corte

5.4. renombrar

Forma	`|> renombrar viejo como nuevo`
Recibe	`viejo`: columna existente. `nuevo`: nombre libre.
Devuelve	La tabla con la columna renombrada, en la misma posición.
Efecto	Los valores no cambian.

Errores: columna inexistente; `no se puede renombrar 'co2' como 'pais': ya existe una columna con ese nombre`.

5.5. ordenar

Forma	`|> ordenar por columna [asc | desc]`
Recibe	Una columna existente. Por defecto, `asc`.
Devuelve	La tabla con las filas ordenadas.
Efecto	Las columnas no cambian.

- Los nulos van siempre **al final**, tanto en orden ascendente como descendente.
- El orden es **estable**: las filas con el mismo valor conservan su orden relativo. Esto permite ordenar por varios criterios aplicando `ordenar` varias veces.
- Los textos se ordenan alfabéticamente.

Errores: columna inexistente; `no se puede ordenar por 'x': la columna mezcla tipos que no se pueden comparar`.

5.6. eliminar duplicados

Forma	`|> eliminar duplicados`
Recibe	Nada.
Devuelve	La tabla sin filas repetidas.
Efecto	Se conserva la primera aparición de cada fila.

Dos filas son iguales si tienen los mismos valores en todas las columnas. Para quitar duplicados según algunas columnas, se seleccionan primero.

5.7. eliminar nulos

Forma	`|> eliminar nulos [en [c1, c2, ...]]`
Recibe	Opcionalmente, una lista de columnas existentes.
Devuelve	La tabla sin las filas que tienen algún nulo en esas columnas.
Efecto	Sin lista, se revisan todas las columnas.

Errores: columna inexistente.

5.8. rellenar nulos

Forma	`|> rellenar nulos en columna con valor`
Recibe	Una columna existente y un valor de celda que no sea nulo.
Devuelve	La tabla con los nulos de esa columna reemplazados.
Efecto	Los valores que no eran nulos no cambian.

Compatibilidad de tipos:

Tipo de la columna	Valor aceptado	Observación
`decimal`	`decimal` o `numero`	El número se guarda como decimal
`numero`	`numero` o `decimal`	
`texto`	`texto`	
`booleano`	`booleano`	
`nulo` o `mixto`	Cualquiera	

Errores:

Situación	Mensaje
Rellenar con nulo	`no se puede rellenar nulos con nulo`
Rellenar con una lista o tabla	`solo se puede rellenar con un numero, decimal, texto o booleano`
Tipo incompatible	`no se puede rellenar la columna 'x' (de tipo decimal) con un valor de tipo texto`

5.9. convertir

Forma	`|> convertir columna como tipo`
Recibe	Una columna existente y un tipo: `numero`, `decimal`, `texto` o `booleano`.
Devuelve	La tabla con los valores de esa columna convertidos.
Efecto	Los nulos siguen siendo nulos.

Destino	Regla
`texto`	Siempre posible. Los booleanos pasan a `"verdadero"` y `"falso"`.
`numero`	De decimal, se descarta la parte decimal. De texto, si el texto es un número. Los booleanos no se convierten.
`decimal`	De número o de un texto numérico. Los booleanos no se convierten.
`booleano`	De los textos `verdadero`, `falso`, `true` y `false`. Los números no se convierten.

Errores: `no se puede convertir el valor 'Colombia' de la columna 'pais' a numero`. La conversión es *todo o nada*: si un valor no se puede convertir, falla la operación completa.

> **Por qué:** Los booleanos no se convierten a números para mantener la separación de tipos del primer corte: en Python `True` vale `1`, pero en Terra un booleano no es un número.

5.10. limitar

Forma	`|> limitar n`
Recibe	Un número entero mayor o igual a 0.
Devuelve	Las primeras `n` filas.
Efecto	Si la tabla tiene menos filas, se devuelven todas.

Errores: `'limitar' necesita un numero entero mayor o igual a 0` (para decimales, negativos, booleanos o textos).

6. Agrupamiento y agregaciones

6.1. agrupar

Forma	`|> agrupar por [c1, c2, ...]`
Recibe	Columnas existentes, sin repetir.
Devuelve	Un valor interno de *tabla agrupada*.
Efecto	Reúne las filas que tienen los mismos valores en esas columnas.

Los grupos aparecen en el orden en que cada combinación de valores aparece por primera vez. Un `nulo` en una columna clave forma su propio grupo.

La tabla agrupada **no es un tipo del lenguaje**: no se puede guardar en una variable ni mostrar. Solo existe como paso intermedio antes de `resumir`.

Errores: columna inexistente; `una columna aparece repetida en 'agrupar por'`; las dos reglas de tuberías sobre `resumir`.

6.2. resumir

Forma	`|> resumir n1 = expr1, n2 = expr2, ...`
Recibe	Una tabla o una tabla agrupada, y una o más agregaciones.
Devuelve	Una tabla nueva.
Efecto	Con `agrupar`: una fila por grupo. Sin `agrupar`: una sola fila.

Columnas del resultado: primero las columnas clave de `agrupar` (si las hay) y después una columna por agregación, con el nombre indicado.

Evaluación. Para cada grupo, las columnas usadas en la expresión se declaran como **la lista de sus valores no nulos** en ese grupo. Por eso las agregaciones se escriben con las mismas funciones que trabajan sobre listas, y se pueden combinar con cualquier expresión:

```
|> resumir rango = maximo(co2) - minimo(co2),
          promedio = redondear(media(co2), 2)
```

Errores:

Situación	Mensaje
La agregación no produce un solo valor	`la agregacion 'x' debe producir un solo valor (por ejemplo con suma, media o contar), se obtuvo lista`
Nombre de agregación repetido o igual a una clave	`la columna 'x' esta repetida`

Situación	Mensaje
Función aplicada a una columna de texto	El error de tipo de la función, por ejemplo al pedir la media de `pais`

6.3. Funciones de agregación

Función	Significado
`contar()`	Número de filas del grupo, incluidas las que tienen nulos. Solo dentro de `resumir`.
`contar(lista)`	Cantidad de elementos. Con una columna, cuenta los valores no nulos.
`suma(lista)`	Suma de los valores.
`media(lista)`	Media aritmética: $\bar{x} = \frac{1}{n}\sum x_i$.
`mediana(lista)`	Valor central de los datos ordenados; con n par, el promedio de los dos centrales.
`minimo(lista)`, `maximo(lista)`	Valor mínimo y máximo.
`desviacion(lista)`	Desviación estándar muestral: $s = \sqrt{\frac{1}{n-1}\sum(x_i - \bar{x})^2}$. Necesita al menos dos valores.

La diferencia entre `contar()` y `contar(columna)` permite medir datos faltantes. En el conjunto de ejemplo, `contar()` da 16 y `contar(energia_renovable)` da 14, porque hay dos nulos.

Error propio de `contar`: `contar() sin argumentos solo puede usarse dentro de 'resumir'`.

7. Estadística descriptiva: describir

Forma	`describir(tabla)`
Recibe	Exactamente una tabla.
Devuelve	Una tabla con una fila por cada columna de la tabla original.
Efecto	Ninguno.

Columna del resultado	Contenido
`columna`, `tipo`	Nombre y tipo de la columna descrita.
`conteo`, `nulos`	Valores presentes y valores nulos.
`media`, `desviacion`, `minimo`, `mediana`, `maximo`	Solo para columnas numéricas.
`distintos`, `mas_frecuente`	Solo para columnas de texto o booleanas. Si hay empate en la frecuencia, gana el valor que aparece primero.

Las estadísticas que no aplican a un tipo de columna quedan como `nulo`, que aquí significa "no aplica". Como el resultado es una tabla, se puede mostrar, guardar o seguir procesando con tuberías.

Errores: `describir() necesita una tabla, se recibio numero` (o `nada`, si no recibe argumentos).

8. mostrar con tablas

`mostrar(tabla)` imprime la tabla alineada: los números a la derecha y los textos a la izquierda, con los nulos como `nulo`. Muestra hasta 10 filas, indica cuántas faltan y termina con el tamaño:

```
pais     | anio |  co2 | poblacion | energia_renovable
---------+------+------+-----------+------------------
Colombia | 2015 | 86.2 |  46700000 |              26.4
Colombia | 2023 | 88.1 |  52300000 |              nulo
... (6 filas mas)
[16 filas x 5 columnas]
```

Los decimales se muestran con un máximo de cuatro cifras y sin ceros innecesarios. El valor guardado no se redondea: el formato solo afecta lo que se imprime.

9. Funciones como valores

9.1. Funciones anónimas

Forma	`funcion(p1, p2, ...) => expresion`
Recibe	Nombres de parámetros distintos y una expresión.
Devuelve	Un valor de tipo `funcion`.
Efecto	Ninguno: el cuerpo no se evalúa hasta que alguien llama a la función.

Al llamarla, los parámetros se declaran como variables no reasignables y el cuerpo se evalúa con **acceso a las variables del programa**, que los parámetros ocultan si tienen el mismo nombre. Al terminar se restaura la memoria.

```
fijo factor = 0.42
fijo emisiones = mapear(consumos, funcion(kwh) => kwh * factor)
```

> **Por qué:** Se reutilizó la palabra reservada `funcion` en lugar de agregar una nueva (como `lambda`), y el símbolo `=>` no choca con ningún operador existente. A diferencia de las funciones con nombre, que solo ven sus parámetros, la anónima ve el programa: así puede usar un factor o umbral declarado afuera, que es su uso más común.

9.2. Funciones con nombre como valores

Escribir el nombre de una función sin paréntesis produce un valor de tipo `funcion`. Funciona con las funciones del usuario y con las incorporadas: `mapear(lista, raiz)`. Una variable que guarda una función se puede llamar como cualquier otra: `doble(21)`.

Reglas y errores:

Situación	Mensaje
Parámetro repetido	`un parametro aparece repetido en la funcion anonima`
Número de argumentos distinto	`la funcion anonima espera 1 argumento(s) y recibio 2`
Se pasa algo que no es función	`mapear(): el segundo argumento debe ser una funcion, por ejemplo funcion(x) => x * 2`

9.3. Funciones de orden superior

Función	Regla
`mapear(lista, f)`	Lista con `f(x)` para cada elemento.
`filtro(lista, f)`	Elementos donde `f(x)` es `verdadero`. `f` debe devolver un booleano.
`reducir(lista, f [, inicial])`	Aplica `f(acumulado, x)` de izquierda a derecha. Sin valor inicial usa el primer elemento; con lista vacía y sin inicial es error.
`ordenar_por(lista, f [, "desc"])`	Ordena según `f(x)`; las claves deben ser comparables.
`todos`, `alguno`	Verdadero si todos o alguno cumplen. `f` debe devolver un booleano.
`componer(f, g)`	Función que calcula `f(g(x))`.
`aplicar(f, a, ...)`	Llama a `f` con esos argumentos.
`aplicar_matriz(M, f)`	Matriz con `f` aplicada a cada celda.
`rango(fin)`, `rango(inicio, fin [, paso])`	Enteros desde `inicio` hasta antes de `fin`. El paso no puede ser 0.

La función de listas se llama `filtro` y no `filtrar` porque `filtrar` es palabra reservada de las tuberías.

10. Matemáticas y matrices

Todas las funciones validan sus argumentos antes de calcular y no usan el módulo `math` de Python.

10.1. Funciones de una variable

Función	Dominio y método
`seno(x)`, `coseno(x)`	Cualquier número (radianes). Se reduce el ángulo a $[-\pi, \pi]$ y se suma la serie de Taylor.
`tangente(x)`	Error donde el coseno vale 0: `tangente() no esta definida en ese angulo`.
`exp(x)`	Error si el resultado es demasiado grande ($x > 709$). Se usa $e^x = (e^{x/2^k})^{2^k}$.
`ln(x)`, `log(x [, base])`	$x > 0$; la base debe ser positiva y distinta de 1. Se escribe $x = m \cdot 2^k$ y $\ln x = k \ln 2 + \ln m$.
`piso`, `techo`	Cualquier número; devuelven un entero.
`factorial(n)`	n entero mayor o igual a 0.
`aleatorio()`, `semilla(n)`	Generador congruencial lineal; con la misma semilla produce la misma secuencia.

Los resultados se redondean a 12 cifras para quitar el ruido de la aritmética de punto flotante: `seno(pi() / 6)` da `0.5` y no `0.49999999999999994`.

10.2. Vectores y matrices

Un **vector** es una lista no vacía de números. Una **matriz** es una lista no vacía de filas no vacías, todas del mismo tamaño y solo con números (los booleanos no cuentan como números).

Regla	Mensaje
Filas de distinto tamaño	`transpuesta(): la matriz tiene filas de distinto tamano`
Producto con dimensiones incompatibles	`mult_matriz(): dimensiones incompatibles 1x2 por 1x2; ...`
Suma de matrices de distinta dimensión	`sumar_matrices(): las matrices deben tener la misma dimension (2x2 y 2x3)`
Determinante, inversa o traza de una matriz no cuadrada	`determinante(): la matriz debe ser cuadrada, es 2x3`
Matriz singular	`inversa(): la matriz no tiene inversa (su determinante es 0)`
Vectores de distinto tamaño	`producto_punto(): los vectores tienen tamanos distintos (2 y 3)`

El determinante se calcula por **eliminación de Gauss con pivoteo parcial** y la inversa por **Gauss-Jordan** sobre la matriz aumentada $[M \mid I]$.

10.3. Número de argumentos

Toda función de la biblioteca verifica cuántos argumentos recibe: `seno() recibe 1 argumento(s) y recibio 0`.

11. Gestor de memoria y recursión

11.1. Límite de recursión

El evaluador cuenta cuántas llamadas a funciones del usuario están abiertas al mismo tiempo. Si pasan del límite (200 por defecto), la ejecución se detiene:

```
Error en evaluacion: [linea 2] la funcion 'infinita' supero el limite de 200 llamadas anidadas; revisa el
    caso base de la recursion
```

Cada llamada de Terra usa muchos niveles de la pila de Python (uno por cada regla de la gramática que se recorre). Por eso el gestor sube el límite de Python lo suficiente para que el de Terra se alcance primero. Si aun así Python se quedara sin pila, el error se convierte en un mensaje de Terra.

Forma	`fijar_limite_recursion(n)`
Recibe	Un entero entre 1 y 2000.
Devuelve	Nulo.
Efecto	Cambia el límite para el resto del programa.

11.2. Consulta y liberación

Función	Regla
`memoria_info()`	Tabla con las variables visibles (nombre, tipo, `fijo` o `var`, elementos y memoria estimada), ordenada de mayor a menor, con una fila de total.
`memoria_estado()`	Tabla con las variables y funciones declaradas, las llamadas anidadas actuales y los límites.
`liberar("x")`	Elimina la variable; después se puede volver a declarar. Error si no existe o si el argumento no es texto.

La memoria se estima recorriendo el valor completo con el tamaño aproximado de cada tipo en Python; no se usa ninguna librería.

Límite de tamaño. Las funciones que crean matrices verifican *antes* de crearlas que no pasen de 5 millones de celdas: `la matriz tendria 100000000 celdas y el limite de memoria es 5000000`.

12. Entrada por teclado

Forma	`leer([mensaje])` / `leer_numero([mensaje])`
Recibe	Un texto opcional que se muestra como pregunta.
Devuelve	`leer`: texto. `leer_numero`: número o decimal.
Efecto	Espera a que el usuario escriba y presione Enter.

`leer_numero` vuelve a preguntar mientras lo escrito no sea un número, para que un error de digitación no detenga el programa. Si no hay más datos de entrada (por ejemplo, cuando se

le pasa un archivo por tubería y se acaba), el error es `no hay mas datos de entrada para leer()`.

13. Tablas, listas y archivos

13.1. De tabla a lista y matriz

Función	Regla
`columna(t, "c")`	Lista con los valores de la columna, incluidos los nulos.
`nombres(t)`, `num_filas(t)`	Nombres de las columnas y número de filas.
`fila(t, i)`	La fila `i`, contando desde 0. Error si no existe.
`valores_unicos(t, "c")`	Valores distintos en el orden en que aparecen.
`tabla_desde(cols, filas)`	Crea una tabla; las celdas deben ser valores de celda y cada fila del tamaño correcto.
`a_matriz(t [, cols])`	Matriz con las columnas indicadas (todas por defecto). Error si hay un texto o un nulo.

13.2. Combinar tablas

Forma	`unir(t1, t2, "clave" [, "interna" | "izquierda"])`
Recibe	Dos tablas que tengan la columna clave.
Devuelve	Una tabla con las columnas de `t1` y las de `t2` sin la clave.
Efecto	Con `"interna"` (por defecto) solo quedan las filas cuya clave está en las dos tablas; con `"izquierda"` quedan todas las de `t1`, con nulos donde no hay pareja.

Si una columna que no es la clave existe en las dos tablas, es error: `unir(): la columna 'anio' existe en las dos tablas; renombrala en una de ellas antes de unir`. Una clave nula nunca encuentra pareja.

`concatenar(t1, t2)` pone las filas de `t2` debajo de las de `t1`. Las dos deben tener las mismas columnas, aunque sea en otro orden.

13.3. Archivos de texto

Función	Regla
`leer_archivo(ruta)`	Todo el contenido como texto.
`leer_lineas(ruta)`	Lista con una línea por elemento, sin el salto de línea.
`escribir_archivo(ruta, texto)`	Crea o reemplaza el archivo.
`agregar_archivo(ruta, texto)`	Agrega al final del archivo.
`existe_archivo(ruta)`	Booleano.
`borrar_archivo(ruta)`	Borra el archivo; error si no existe.

Los errores de archivo indican la función y la ruta, por ejemplo `leer_archivo(): no se encontro el archivo o la carpeta de 'no_existe.txt'`.

13.4. Secuencias de escape en los textos

Dentro de un texto, `\n` es un salto de línea, `\t` un tabulador, `\"` una comilla y `\\` una barra invertida.

13.5. Tuberías dentro de argumentos

Los argumentos de una función, igual que la tabla de `guardar`, son expresiones *sin* tubería. Esta regla viene del primer corte y evita que `|>` absorba lo que sigue. Una tubería se puede usar ahí si va entre paréntesis:

```
mostrar(unir((datos |> filtrar donde anio == 2021), metas, "pais"))
```

14. Ayuda integrada

`ayuda()` lista las categorías del lenguaje, `ayuda("matrices")` muestra cómo se escribe cada función de una categoría con un ejemplo, y `ayuda("mapear")` muestra una función concreta. Si el tema no existe: `no hay ayuda para 'xyz'. Escribe ayuda() para ver las categorias`.

15. Formato de los errores

Todos los errores semánticos detienen la ejecución y se muestran con el mismo formato, sin traza de Python:

```
Error en evaluacion: [linea 2] la columna 'temperatura' no existe. Columnas disponibles: pais, anio, co2,
    poblacion, energia_renovable
```

Internamente hay dos clases de error:

- `ErrorDatos`, que lanzan `tabla.py` y `operaciones.py`. Estos módulos no conocen el árbol de ANTLR, así que no saben en qué línea del programa están.
- `ErrorSemantico`, que lanza el Visitor. Cuando atrapa un `ErrorDatos`, lo convierte en `ErrorSemantico` agregando la línea de la instrucción que lo causó.

> **Por qué:** Así el procesamiento de datos queda separado del lenguaje, y aun así cada mensaje llega al usuario con la línea exacta donde está el problema.

15.1. Resumen de las categorías de error

Categoría	Ejemplos
Identificadores	Variable no declarada, redeclarada o constante reasignada; función inexistente.
Columnas	Columna inexistente, repetida o con un nombre ya ocupado.
Tipos incompatibles	Ruta que no es texto, condición no booleana, relleno con otro tipo, conversión imposible, tubería que no empieza con tabla.
Operaciones no permitidas	Operación tras `agrupar`, tubería sin `resumir`, `contar()` fuera de `resumir`, `limitar` negativo, opción repetida.
Archivos	Archivo inexistente, vacío, mal formado, con otro separador o sin permisos.
Matemáticas	Fuera del dominio (`ln(0)`), dimensiones incompatibles, matriz singular.
Recursos	Límite de recursión superado, matriz demasiado grande, variable inexistente al liberar.
Funciones	Algo que no es función, número de argumentos distinto, función que no devuelve booleano en `filtro`.